Cyberscope

A TAC Security Company

Audit Report

Five Pillars Vesting Wallet

May 2026

Files: VestingWallet.sol

Sha256:

4cac474b78f7680bb58e67c74e268bf2243d04143c09b4a1d627c6951fdbaa0e

Audited by © cyberscope

Table of Contents

Risk Classification

The criticality of findings in Cyberscope's smart contract audits is determined by evaluating multiple variables. The two primary variables are:

1. **Likelihood of Exploitation**: This considers how easily an attack can be executed, including the economic feasibility for an attacker.
2. **Impact of Exploitation**: This assesses the potential consequences of an attack, particularly in terms of the loss of funds or disruption to the contract's functionality.

Based on these variables, findings are categorized into the following severity levels:

1. **Critical**: Indicates a vulnerability that is both highly likely to be exploited and can result in significant fund loss or severe disruption. Immediate action is required to address these issues.
2. **Medium**: Refers to vulnerabilities that are either less likely to be exploited or would have a moderate impact if exploited. These issues should be addressed in due course to ensure overall contract security.
3. **Minor**: Involves vulnerabilities that are unlikely to be exploited and would have a minor impact. These findings should still be considered for resolution to maintain best practices in security.
4. **Informative**: Points out potential improvements or informational notes that do not pose an immediate risk. Addressing these can enhance the overall quality and robustness of the contract.

Severity	Likelihood / Impact of Exploitation
Critical	Highly Likely / High Impact
Medium	Less Likely / High Impact or Highly Likely/ Lower Impact
Minor / Informative	Unlikely / Low to no Impact

Review

Audit Updates

Initial Audit	03 May 2026

Source Files

Filename	SHA256
VestingWallet.sol	4cac474b78f7680bb58e67c74e268bf2243d04143c09b4a1d627c6951fdbaa0e

Overview

VestingWallet.sol is a Solidity smart contract that implements a 12-year linear vesting schedule with daily token release for a single beneficiary. The contract is compact and well-structured, making appropriate use of SafeERC20, custom errors, immutables, and modern Solidity patterns. This report presents a comprehensive security audit of the contract, covering security, correctness, access control, edge cases, and design assumptions across all components.

Findings Breakdown

Critical	0
Medium	0
Minor / Informative	8

Severity	Unresolved	Acknowledged	Resolved	Other
Critical	0	0	0	0
Medium	0	0	0	0
Minor / Informative	8	0	0	0

Diagnostics

Critical Medium Minor / Informative

Severity	Code	Description	Status
	AME	Address Manipulation Exploit	Unresolved
	IVAL	Improper Vesting Allocation Logic	Unresolved
	I1D	Inaccurate 12-Year Duration	Unresolved
	PSU	Potential Subtraction Underflow	Unresolved
	RAZA	Release Allows Zero Amount	Unresolved
	URF	Unrestricted Release Function	Unresolved
	VCPL	Vesting Calculation Precision Loss	Unresolved
	L19	Stable Compiler Version	Unresolved

AME - Address Manipulation Exploit

Criticality	Minor / Informative
Location	VestingWallet.sol#L86
Status	Unresolved

Description

The contract's design includes functions that accept external contract addresses as parameters without performing adequate validation or authenticity checks. This lack of verification introduces a significant security risk, as input addresses could be controlled by attackers and point to malicious contracts. Such vulnerabilities could enable attackers to exploit these functions, potentially leading to unauthorized actions or the execution of malicious code under the guise of legitimate operations.

```
function release(address token) public virtual {
    uint256 amount = releasable(token);
    ...
}
```

Recommendation

To mitigate this risk and enhance the contract's security posture, it is imperative to incorporate comprehensive validation mechanisms for any external contract addresses passed as parameters to functions. This could include checks against a whitelist of approved addresses, verification that the address implements a specific contract interface or other methods that confirm the legitimacy and integrity of the external contract. Implementing such validations helps prevent malicious exploits and ensures that only trusted contracts can interact with sensitive functions.

IVAL - Improper Vesting Allocation Logic

Criticality	Minor / Informative
Location	VestingWallet.sol#L98
Status	Unresolved

Description

The vestedAmount() function reads the contract's live token balance at call time rather than a fixed amount set at deployment. This means any tokens sent to the contract after deployment are immediately added to the vesting base, allowing the beneficiary to claim more than intended — either by accident or deliberately — at any point during the schedule.

```
uint256 totalAllocation = IERC20(token).balanceOf(address(this)) +
released(token);
```

Recommendation

Record the total token allocation once at deployment or at a designated funding point, and use that fixed number as the basis for all vesting calculations. Tokens sent after that point should not affect the schedule — they should either be rejected or handled through a separate process.

I1D - Inaccurate 12-Year Duration

Criticality	Minor / Informative
Location	VestingWallet.sol#L24
Status	Unresolved

Description

The contract calculates 12 years as exactly 4,380 days, ignoring leap years. In reality, a 12-year period includes 3 leap years, making the true duration 4,383 days. As a result, the vesting schedule ends 3 days earlier than the natural 12-year anniversary — which, while financially harmless, could cause confusion or disputes with stakeholders who expect an exact 12-year term.

```
  uint64 private constant _duration = 60 * 60 * 24 * 365 * 12; // 12
years
```

Recommendation

If calendar accuracy is important, adjust the duration constant to account for leap years. At minimum, document this approximation clearly in the contract notes and any legal or token allocation agreements so all parties are aware upfront.

PSU - Potential Subtraction Underflow

Criticality	Minor / Informative
Location	VestingWallet.sol#L78
Status	Unresolved

Description

The releasable() function performs a subtraction between vestedAmount() and released(token). If a rebasing or non-standard token is used, the contract's live balance could decrease unexpectedly, causing vestedAmount() to return a value smaller than released(token). As a result, the subtraction may underflow and cause the execution to revert.

```
    return vestedAmount(token, uint64(block.timestamp)) -
released(token);
```

Recommendation

The team is advised to properly handle the subtraction in releasable() to avoid potential underflow and ensure the reliability and safety of the contract. The contract should ensure that vestedAmount() is always greater than or equal to released(token) before performing the subtraction, and should not allow execution of this section if the condition is violated.

RAZA - Release Allows Zero Amount

Criticality	Minor / Informative
Location	VestingWallet.sol#L86
Status	Unresolved

Description

The release() function can be called even when there are no tokens to release. When this happens, it records a zero amount, emits a ERC20Released(token, 0) event, and executes a zero-value transfer — all unnecessarily. Any off-chain system tracking these events will log a phantom release, polluting the audit trail and potentially causing incorrect accounting in integrated tools.

```
function release(address token) public virtual {
        uint256 amount = releasable(token);
        _erc20Released[token] += amount;
        emit ERC20Released(token, amount);
        SafeERC20.safeTransfer(IERC20(token), beneficiary, amount);
    }
```

Recommendation

Add a check at the start of release() that stops execution if there is nothing to release. This prevents wasted gas, misleading events, and unnecessary external calls.

URF - Unrestricted Release Function

Criticality	Minor / Informative
Location	VestingWallet.sol#L86
Status	Unresolved

Description

The release() function is public and can be called by anyone, not just the beneficiary. While tokens always transfer to the immutable beneficiary address and cannot be redirected, the beneficiary has no control over when releases are triggered, as any external party can initiate them at any time.

```
    function release(address token) public virtual {
```

Recommendation

Restrict the release() function so that only the beneficiary can call it, giving them full control over when tokens are claimed.

VCPL - Vesting Calculation Precision Loss

Criticality	Minor / Informative
Location	VestingWallet.sol#L115
Status	Unresolved

Description

The _vestingSchedule() function uses integer division to calculate the daily releasable amount, which lose any remainder on each calculation. Over the full 12-year schedule this results in a small cumulative precision loss, though the contract's own notes acknowledge this and estimate the maximum loss at ~0.02% of the total allocation.

```
    return (totalAllocation * daysPassed) / totalDays;
```

Recommendation

Consider using a higher precision calculation method to minimize the cumulative rounding loss over the full vesting duration.

L19 - Stable Compiler Version

Criticality	Minor / Informative
Location	VestingWallet.sol#L2
Status	Unresolved

Description

The ^ symbol in the pragma allows the contract to be compiled with any compatible Solidity version above 0.8.28. This means different deployments could use different compiler versions, potentially introducing unexpected behavior or undiscovered bugs.

```
pragma solidity ^0.8.28;
```

Recommendation

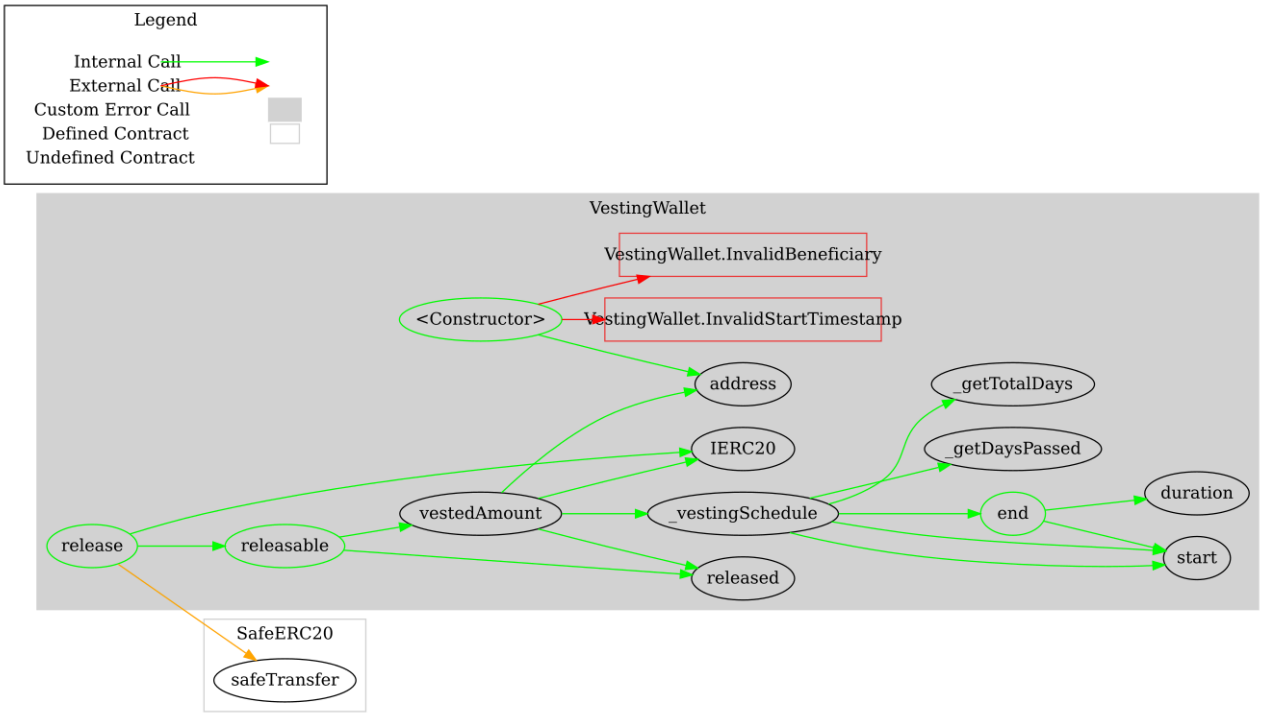
Lock the pragma to the exact compiler version used during development and testing to ensure consistent and predictable compilation across all environments.

Functions Analysis

Contract	Type	Bases		
	Function Name	**Visibility**	**Mutability**	**Modifiers**
VestingWallet	Implementation			
		Public	✓	-
	start	Public		-
	duration	Public		-
	end	Public		-
	released	Public		-
	releasable	Public		-
	release	Public	✓	-
	vestedAmount	Public		-
	_vestingSchedule	Internal		
	_getDaysPassed	Internal		
	_getTotalDays	Internal		

Inheritance Graph

Flow Graph


Legend
Internal Call
External Call
Custom Error Call
Defined Contract
Undefined Contract
VestingWallet
VestingWallet.InvalidBeneficiary
<Constructor>
VestingWallet.InvalidStartTimestamp
address
_getTotalDays
IERC20
_getDaysPassed
vestedAmount
_vestingSchedule
end
duration
release
releasable
start
released
SafeERC20
safeTransfer

Summary

Five Pillars Token contract implements a vesting mechanism. This audit investigates security issues, business logic concerns and potential improvements.

.

Disclaimer

The information provided in this report does not constitute investment, financial or trading advice and you should not treat any of the document's content as such. This report may not be transmitted, disclosed, referred to or relied upon by any person for any purposes nor may copies be delivered to any other person other than the Company without Cyberscope's prior written consent. This report is not nor should be considered an "endorsement" or "disapproval" of any particular project or team. This report is not nor should be regarded as an indication of the economics or value of any "product" or "asset" created by any team or project that contracts Cyberscope to perform a security assessment. This document does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors' business, business model or legal compliance. This report should not be used in any way to make decisions around investment or involvement with any particular project. This report represents an extensive assessment process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk Cyberscope's position is that each company and individual are responsible for their own due diligence and continuous security Cyberscope's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies and in no way claims any guarantee of security or functionality of the technology we agree to analyze. The assessment services provided by Cyberscope are subject to dependencies and are under continuing development. You agree that your access and/or use including but not limited to any services reports and materials will be at your sole risk on an as-is where-is and as-available basis Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives false negatives and other unpredictable results. The services may access and depend upon multiple layers of third parties.

About Cyberscope

Cyberscope is a TAC blockchain cybersecurity company that was founded with the vision to make web3.0 a safer place for investors and developers. Since its launch, it has worked with thousands of projects and is estimated to have secured tens of millions of investors' funds.

Cyberscope is one of the leading smart contract audit firms in the crypto space and has built a high-profile network of clients and partners.

A **TAC Security** Company

The Cyberscope team

cyberscope.io